



CHAIN OF RESPONSIBILITY

(Mô hình chuỗi trách nhiệm)

1. Mục tiêu bài học

Sau tuần này, sinh viên có thể:

- Hiểu bản chất CoR (tránh nhầm lẫn với if-else lồng nhau)
- Thiết kế Order Processing Pipeline linh hoạt
- Refactor OrderService thành Orchestrator
- Dễ dàng:
 - Thêm bước xử lý mới (FraudCheck, Logger...)
 - Thay đổi thứ tự xử lý
 - Bật/tắt bước xử lý (Dynamic Pipeline)

2. Bối cảnh thực tế – Pipeline

Một đơn hàng khi checkout thường trải qua:

1. Validate order (Dữ liệu đúng không?)
2. Check inventory (Còn hàng không?)
3. Apply discount (Giảm giá voucher)
4. Process payment (Thanh toán tiền)
5. Create shipment (Tạo vận đơn)
6. Send notification (Gửi mail)

🔗 Câu hỏi thiết kế:

- Nếu thêm bước "Fraud Check" thì sửa bao nhiêu file?
- Nếu đổi thứ tự xử lý thì sao?
- Nếu chỉ bật một số bước theo loại order?

3. Thiết kế ban đầu – VI PHẠM

✗ Code dạng Pipeline cứng. Vi phạm SRP & OCP nghiêm trọng.

```
public class OrderService {  
    public void Process(Order order) {  
        Validate(order);  
        CheckInventory(order);  
        ApplyDiscount(order);  
        ProcessPayment(order);  
        CreateShipment(order);  
        SendNotification(order);  
    }  
}
```

4. Phân tích vấn đề

Vấn đề	Giải thích
Vi phạm SRP	OrderService làm quá nhiều việc, biết quá nhiều bước.
Vi phạm OCP	Thêm bước mới -> Phải sửa code hàm Process().
Khó Test	Phải mock toàn bộ các bước để test 1 luồng.
Thứ tự cứng	Không thể cấu hình pipeline runtime.

5. Ý tưởng Chain of Responsibility

👉 Ý tưởng cốt lõi:

Tách mỗi bước thành 1 Object riêng biệt (Handler).

Các Handler được nối với nhau thành chuỗi (Linked List).

Handler:

1. Xử lý phần việc của mình.
2. Gọi handler tiếp theo (nếu có).

👉 Client chỉ cần gọi Handler đầu tiên. Không cần biết cả chuỗi.

6. Base Handler (Template)

```
public abstract class OrderHandler : IOrderHandler
{
    private IOrderHandler _next;

    public void SetNext(IOrderHandler next) {
        _next = next;
    }

    public virtual void Handle(Order order) {
        // Template Method: Xử lý xong thì gọi next
        _next?.Handle(order);
    }
}
```

7. Concrete Handlers

```
public class ValidateHandler : OrderHandler {  
    public override void Handle(Order order) {  
        if (!order.IsValid) throw new Exception("Invalid");  
        Console.WriteLine("Validate OK");  
        base.Handle(order); // Gọi next  
    }  
}
```

```
public class InventoryHandler : OrderHandler {  
    public override void Handle(Order order) {  
        Console.WriteLine("Check Inventory");  
        base.Handle(order);  
    }  
}
```

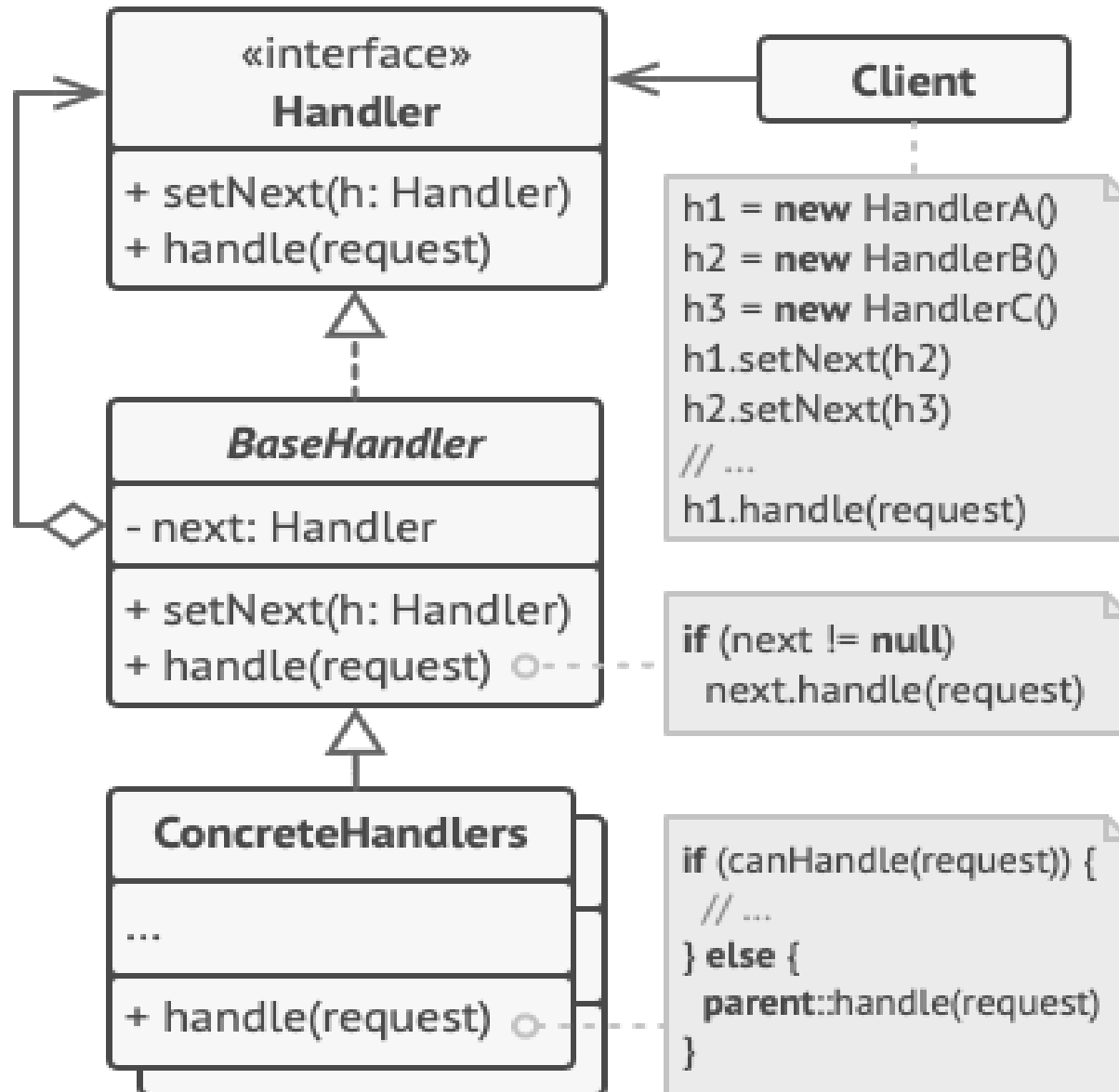

8. Lắp ráp Pipeline (Composition Root)

```
// Setup Pipeline (Tại Startup/Config)
var validate = new ValidateOrderHandler();
var inventory = new InventoryHandler();
var payment = new PaymentHandler();
var notify = new NotificationHandler();
```

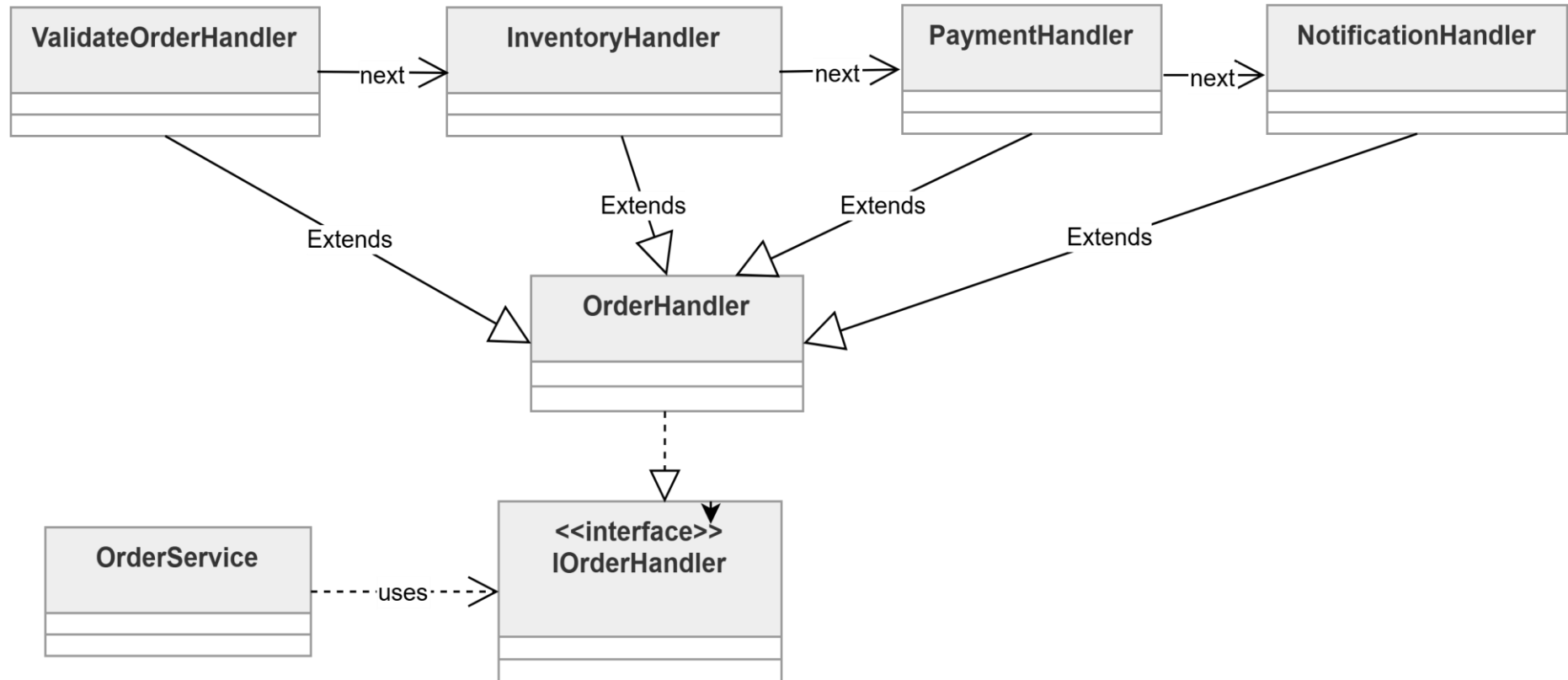
```
// Nối chuỗi
validate.SetNext(inventory);
inventory.SetNext(payment);
payment.SetNext(notify);
```

```
// Usage
var service = new OrderService(validate); // Chỉ cần biết đầu chuỗi
service.Process(order);
```

9. Structure of Chain of Responsibility



10. Chain of Responsibility: Case Study



11. So sánh Strategy vs CoR

Tiêu chí	Strategy Pattern	Chain of Responsibility
Cấu trúc	Phẳng (1 lựa chọn)	Chuỗi (Nhiều bước)
Luồng đi	Chọn 1 trong N thuật toán	Đi qua 1, 2 hoặc tất cả bước
Ví dụ	Chọn phương thức thanh toán	Quy trình xử lý đơn hàng

12. Liên hệ: Middleware ASP.NET

💡 CoR chính là "Middleware" trong ASP.NET Core!

```
app.UseAuthentication();  
app.UseAuthorization();  
app.UseStaticFiles();  
app.UseEndpoints(...);
```

📌 Request đi qua từng middleware.

📌 Mỗi middleware có thể chặn request hoặc gọi next().

13. Bài tập thực hành



Bài 1:

Thêm FraudCheckHandler vào đầu pipeline.

Nếu phát hiện gian lận -> Dừng pipeline, ném lỗi.



Bài 2:

Tạo pipeline động:

- Nếu Order > 10 triệu -> Thêm bước "ManagerApproval".
- Nếu Order Digital -> Bỏ bước "Shipment".



Bài 3:

Vẽ UML Diagram cho trường hợp trên.

14. Kết luận

“ Order không phải là một hành động.
Đó là một chuỗi trách nhiệm. ”

“ Chain of Responsibility giúp code bền vững
khi quy trình nghiệp vụ thay đổi liên tục. ”